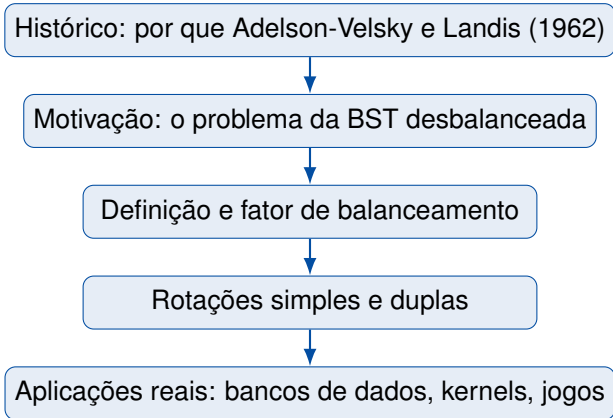


Árvores AVL

Equilíbrio que garante $O(\log n)$

Prof. Andre Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br



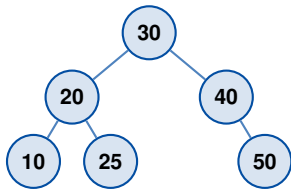
Histórico

Quem inventou

Em **1962**, os matemáticos soviéticos **Georgy Adelson-Velsky** e **Evgenii Landis** publicaram, no *Doklady Akademii Nauk SSSR*, a estrutura que ficou conhecida por suas iniciais: **AVL**.

Por que importou

Foi a *primeira* árvore de busca binária auto-balanceada, garantindo busca, inserção e remoção em tempo $O(\log n)$ no *pior caso*.



Uma AVL equilibrada

- Memória: **KB**, não GB. Tempo de CPU caríssimo.
- Pesquisa em listas ordenadas $\Rightarrow O(n)$ inaceitável.
- BSTs já existiam, mas *degradavam* para listas ligadas.
- Adelson-Velsky e Landis perguntaram: “*Podemos manter o equilíbrio com custo constante por inserção?*”

Curiosidade

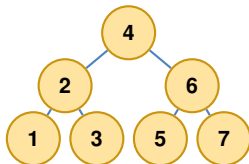
A AVL (1962) veio antes da B-Tree (Bayer e McCreight, 1970–72) e das árvores Rubro-Negras (cujo precursor, a B-tree binária simétrica de Bayer, é de 1972; a forma rubro-negra foi formalizada por Guibas e Sedgewick em 1978).

Motivação

Caso bom — BST equilibrada

Altura $h \approx \log_2 n$

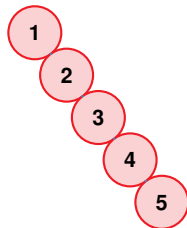
Busca: $O(\log n)$



Caso ruim — inserção ordenada

Altura $h = n$

Busca: $O(n)$ (degenerada!)



Buscar 1 elemento em $n = 1.000.000$ de chaves

Estrutura	Comparações	Tempo (1ns/op)
Lista ligada	1.000.000	1,0 ms
BST degenerada	1.000.000	1,0 ms
AVL equilibrada	≈ 20	0,00002 ms

Ganho

50.000 vezes mais rápido — e a diferença *cresce* com o tamanho dos dados.

Definição

Definição (Adelson-Velsky e Landis, 1962)

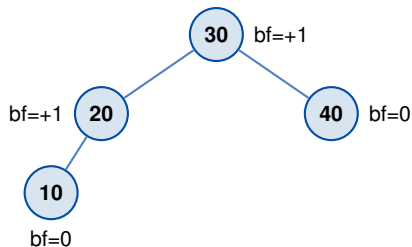
Convenção de altura: a altura de um nó é o **número de nós** no caminho mais longo dele até uma folha; uma folha tem altura 1 e a sub-árvore vazia tem altura 0. (Atenção: aqui contamos **nós**, não arestas — difere da aula de BST, onde uma folha tinha altura 0.)

Uma **árvore AVL** é uma BST em que, para *todo* nó v ,

$$\text{bf}(v) = h(\text{sub-árvore esquerda de } v) - h(\text{sub-árvore direita de } v) \in \{-1, 0, +1\}.$$

Intuição

Em *cada* nó, os dois lados podem diferir em **no máximo 1** nível de altura. Isso impede que a árvore vire “galho torto”.

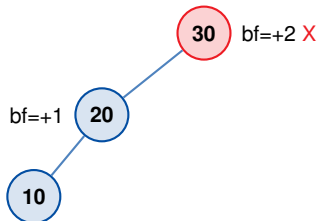


Calculando bf

$$\begin{aligned}bf(20) &= h(\text{esq}) - h(\text{dir}) \\ &= 1 - 0 = +1 \checkmark\end{aligned}$$

$$bf(30) = 2 - 1 = +1 \checkmark$$

Todos $\in \{-1, 0, +1\}$:
é uma AVL válida.



Violação

Após inserir o 10, o nó 30 tem altura esquerda = 2 e direita = 0: $bf(30) = +2$.
Precisamos rebalancear.

Teorema

A altura h de uma AVL com n nós satisfaz

$$h \leq 1,44 \cdot \log_2(n + 2) + 0,672.$$

Consequência prática

Para $n = 10^9$ chaves, $h < 45$. *Toda* busca, inserção ou remoção visita no máximo 45 nós.

Rotações

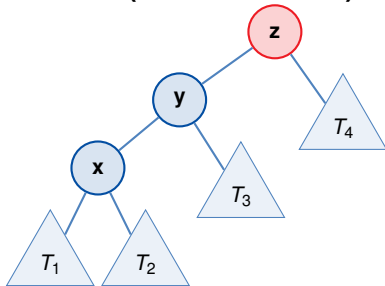
Ideia

Quando a inserção viola o balanceamento, aplicamos uma **rotação** local: *dois nós* trocam de posição (e uma sub-árvore é reatribuída), preservando a ordem da BST e restaurando $|bf| \leq 1$.

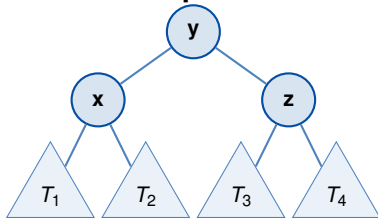
Há **quatro casos**, classificados pelo *caminho* do nó inserido a partir do desequilíbrio:

- **Esq-Esq** $\Rightarrow$ rotação simples à direita
- **Dir-Dir** $\Rightarrow$ rotação simples à esquerda
- **Esq-Dir** $\Rightarrow$ rotação dupla esq-dir
- **Dir-Esq** $\Rightarrow$ rotação dupla dir-esq

Antes (desbalanceada)

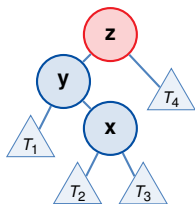


Depois

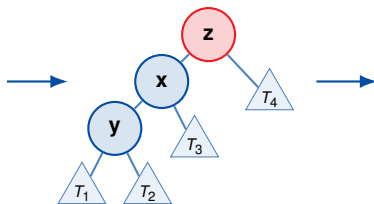


Custo

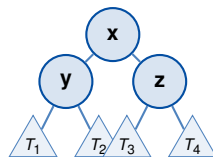
$O(1)$: apenas **três ponteiros** são reatribuídos.



1. desbalanceio Esq-Dir



2. rotação à esq em y



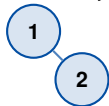
3. rotação à dir em z

Receita (nesta ordem)

Esq-Dir: 1) rotação à **esquerda** no filho esquerdo y; 2) rotação à **direita** no nó desbalanceado z.

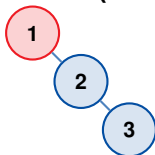
Exemplo: inserindo 1, 2, 3 em sequência

Inserir 1, 2



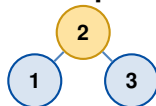
bf válido

Inserir 3 (viola)



$bf(1) = -2$

Rot. esquerda



equilibrada ✓

O que aprendemos

- Rotação é uma operação *local* de custo $O(1)$.
- Quatro casos $\Rightarrow$ duas rotações simples e duas duplas.
- A ordem da BST é **sempre** preservada.
- No máximo *uma* rotação após inserção; até $O(\log n)$ após remoção.

Insertão

Algoritmo em duas fases

1. **Fase BST:** inserir a chave como em uma BST comum, recursivamente, até encontrar uma folha.
2. **Fase de retorno:** subindo pela pilha de chamadas, atualizar a altura de cada ancestral e, se $|bf| > 1$, aplicar a rotação correspondente.

Custo

Busca da posição: $O(\log n)$.

Atualização e rebalanceamento: $O(\log n)$ no caminho de volta.

No máximo uma rotação (simples ou dupla) é necessária por inserção.

Classificação pelo caminho do novo nó

Condição	Caso	Rotação
$bf > 1$ e $chave < filho-esq.chave$	Esq-Esq	dir. simples
$bf < -1$ e $chave > filho-dir.chave$	Dir-Dir	esq. simples
$bf > 1$ e $chave > filho-esq.chave$	Esq-Dir	esq. no filho, dir. no nó
$bf < -1$ e $chave < filho-dir.chave$	Dir-Esq	dir. no filho, esq. no nó

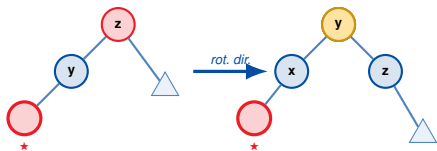
Regra prática

O nome do caso indica os **dois primeiros passos do caminho** do nó desbalanceado até o nó recém-inserido.

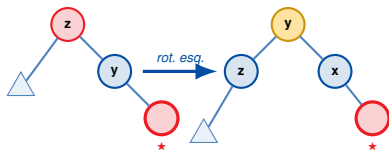
Os 4 casos, visualmente

★ = nó recém-inserido vermelho = nó desbalanceado ($|bf| = 2$) amarelo = raiz após rebalancear

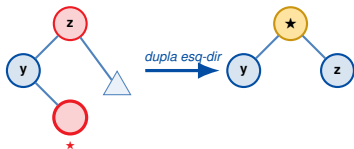
Esq-Esq



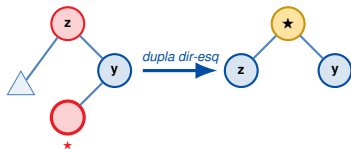
Dir-Dir



Esq-Dir



Dir-Esq

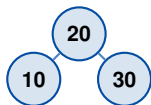


```
1 def insert(node, key):
2     if node is None: return AVLNode(key)
3     if key < node.key: node.left = insert(node.left, key)
4     elif key > node.key: node.right = insert(node.right, key)
5     else: return node # duplicata
6
7     node.height = 1 + max(h(node.left), h(node.right))
8     bf = h(node.left) - h(node.right)
9
10    if bf > 1 and key < node.left.key: return rotate_right(node) # EE
11    if bf < -1 and key > node.right.key: return rotate_left(node) # DD
12    if bf > 1 and key > node.left.key: # ED
13        node.left = rotate_left(node.left); return rotate_right(node)
14    if bf < -1 and key < node.right.key: # DE
15        node.right = rotate_right(node.right); return rotate_left(node)
16    return node
```

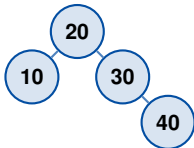

Exemplo: inserir 10, 20, 30, 40, 50

Após 10, 20, 30

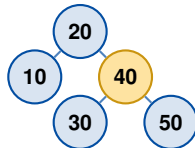
(rot. esq. em 10)



Inserir 40 (ok)



Inserir 50 (DD em 30)



Observação

A rotação resolve o desbalanceio e a altura do nó volta ao valor anterior à inserção — por isso a propagação cessa.

O que aprendemos

- Inserção AVL = inserção BST + rebalanceamento na volta.
- Quatro casos, identificados por bf e pela comparação da chave com o filho.
- Custo total $O(\log n)$, com no máximo uma rotação.

Remoção

Algoritmo em três fases

1. **Remoção BST:** localizar o nó e removê-lo (folha, um filho, ou substituir pelo *sucessor in-order* quando há dois filhos).
2. **Atualização de altura:** ao retornar pela pilha, recalcular a altura de cada ancestral.
3. **Rebalanceamento:** se $|bf| > 1$, aplicar a rotação adequada **e continuar subindo**, pois a altura pode ter mudado.

Diferença crítica vs. inserção

Uma única rotação *não* basta: a remoção pode exigir até $O(\log n)$ rotações em cascata ao longo do caminho.

Classificação pelo bf do filho “mais alto”

Condição	Caso	Rotação
$bf > 1$ e $bf(\text{filho-esq}) \geq 0$	Esq-Esq	dir. simples
$bf < -1$ e $bf(\text{filho-dir}) \leq 0$	Dir-Dir	esq. simples
$bf > 1$ e $bf(\text{filho-esq}) < 0$	Esq-Dir	dupla esq-dir
$bf < -1$ e $bf(\text{filho-dir}) > 0$	Dir-Esq	dupla dir-esq

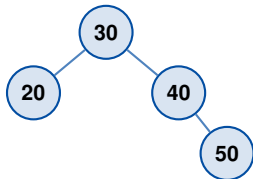
Por que mudou o critério?

Na inserção, o novo nó indica de que lado o desequilíbrio veio. Na remoção, não há “nó novo” — inspecionamos o bf do filho oposto ao lado encurtado.

```
1 def delete(node, key):
2     if node is None: return None
3     if key < node.key: node.left = delete(node.left, key)
4     elif key > node.key: node.right = delete(node.right, key)
5     else:
6         if node.left is None: return node.right # 0 ou 1 filho
7         if node.right is None: return node.left
8         succ = min_node(node.right) # sucessor in-order
9         node.key = succ.key
10        node.right = delete(node.right, succ.key)
11
12    node.height = 1 + max(h(node.left), h(node.right))
13    bf = h(node.left) - h(node.right)
14
15    if bf > 1 and bf2(node.left) >= 0: return rotate_right(node) # EE
16    if bf > 1 and bf2(node.left) < 0:
17        node.left = rotate_left(node.left); return rotate_right(node) # ED
18    if bf < -1 and bf2(node.right) <= 0: return rotate_left(node) # DD
19    if bf < -1 and bf2(node.right) > 0:
20        node.right = rotate_right(node.right); return rotate_left(node) # DE
21    return node
```

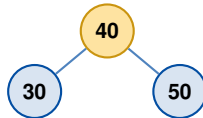
Exemplo: remover 20 da árvore

Antes (AVL válida)



$$\text{bf}(30) = 1 - 2 = -1 \checkmark$$

Remove 20 $\Rightarrow$ rot. esq. em 30



$$\text{Após remover 20: } \text{bf}(30) = 0 - 2 = -2$$

Caso Dir-Dir

O filho mais alto de 30 é o 40, com $\text{bf}(40) = -1 \leq 0 \Rightarrow$ rotação simples à **esquerda** em 30.

O que aprendemos

- Remoção BST padrão + substituição pelo sucessor in-order.
- Critério dos 4 casos usa o bf do filho mais alto.
- Pode exigir **múltiplas rotações em cascata** ao subir.
- Custo total ainda é $O(\log n)$.

Aplicações

Sistemas reais

- **Kernel Linux:** árvores Rubro-Negras (`rbtree.h`) escalonam processos.
- **PostgreSQL / MySQL:** índices B-Tree (parente da AVL para disco).
- **C++ STL:** `std::map`, `std::set`.
- **Java:** `TreeMap`, `TreeSet`.
- **Redis:** Sorted Sets (skip lists, mesma motivação).

Casos de uso

- Roteamento de pacotes em redes
- Detecção de colisões em motores de jogo
- Autocompletar e dicionários
- Bancos de dados geoespaciais
- Bibliotecas de *intervalos* (calendários)

Três razões

1. **Modelo mental:** entender *trade-offs* de altura vs. custo de rebalanceamento.
2. **Porta de entrada:** Rubro-Negra, B-Tree, Treap, Splay — todas derivam da mesma ideia.
3. **Entrevistas e olimpíadas:** aparece em *quase toda* entrevista de empresa de tecnologia.

Mensagem final

A AVL é uma das ideias mais elegantes da computação: *disciplina local* produz *garantia global*.

Você agora sabe

- Por que BSTs ingênuas falham e quem resolveu (1962).
- Definir o fator de balanceamento e identificar violações.
- Aplicar as 4 rotações para restaurar o equilíbrio.
- Onde AVL (e parentes) sustentam sistemas reais.

Próximos passos

Tutorial Jupyter: implementar inserção AVL em Python e visualizar a árvore após cada operação.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

Introduction to Algorithms.

MIT Press, 3 edition, 2009.



Robert W. Floyd.

Algorithm 245: Treesort 3.

Communications of the ACM, 7(12):701, 1964.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4 edition, 2011.



J. W. J. Williams.

Algorithm 232: Heapsort.

Communications of the ACM, 7(6):347–348, 1964.



Nivio Ziviani.

Projeto de Algoritmos: com Implementações em Pascal e C.

Cengage Learning, 3 edition, 2011.